



Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 Redes

Laboratorio 2

Esquemas de detección y corrección de errores

1 Antecedentes

El ruido y los errores de transmisión ocurren en toda comunicación, y es parte de los retos al implementar este tipo de sistemas manejar adecuadamente las fallas que puedan ocurrir. Por lo tanto, a lo largo de la evolución del Internet se han desarrollado distintos mecanismos que sirven tanto para la detección como para la corrección de errores.

2 Objetivos

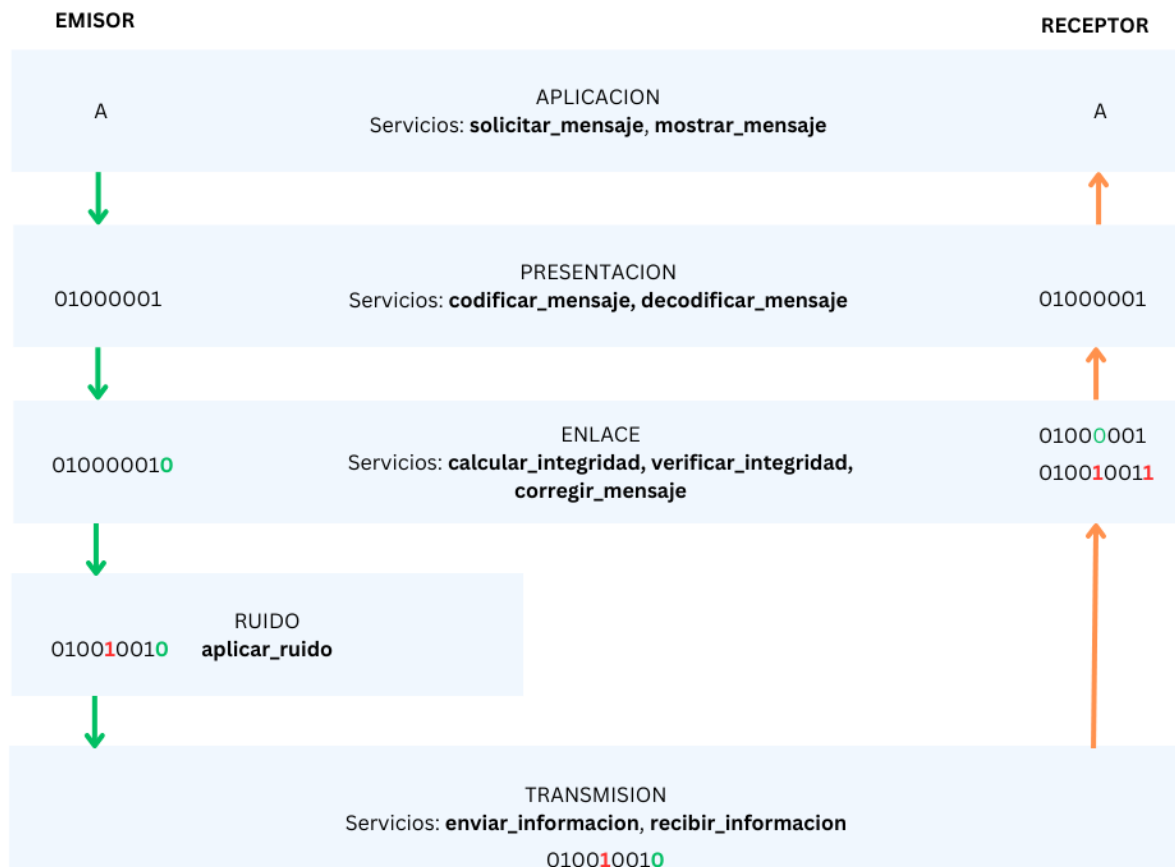
- Comprender el funcionamiento de un modelo de capas y sus servicios
- Implementar los algoritmos de detección y corrección de errores.
- Analizar el funcionamiento de los esquemas de detección y corrección
- Experimentar la transmisión de información expuesta a un canal no confiable
- Identificar las ventajas y desventajas de cada uno de los algoritmos.

3 Desarrollo

En clase estudiamos que entre los servicios que ofrece la capa de Enlace se encuentran la detección y corrección de errores, pues se asume que el medio en el que se transmite la data no es confiable. En este laboratorio se implementará al menos un algoritmo de cada uno de ellos y una aplicación para la transmisión y recepción de mensajes, con base a una arquitectura de capas con distintos servicios. El laboratorio se trabajará en parejas y, si el número de estudiantes es impar, habrá un único trío.

a. Arquitectura de capas

La arquitectura cuenta con las siguientes capas y servicios:



Descripción de los servicios:

1. APLICACIÓN

- **Solicitar mensaje**: solicita el texto a enviar al emisor. También solicita el algoritmo a utilizar para comprobar la integridad.
- **Mostrar mensaje**: muestra el mensaje al receptor (sin errores). Si se detectaron errores y no fue posible corregirlos, se debe indicar con un mensaje de error.

2. PRESENTACION

- **Codificar mensaje**: codifica cada carácter en ASCII binario. Por ejemplo, para el carácter A el código binario ASCII es 01000001.
- **Decodificar mensaje**: si no se detectan errores, se debe decodificar el ASCII binario a los caracteres correspondientes. Si se detecta un error, se debe indicar de alguna forma a la capa de aplicación.

3. ENLACE

- **Calcular integridad**: utilizando el algoritmo indicado en el servicio **solicitar_mensaje**, calcular la información de integridad. Concatenar la información al mensaje binario original.

- **Verificar_integridad:** el algoritmo seleccionado debe calcular la información del lado del receptor y compararla contra la proporcionada por el emisor para detectar posibles errores. Debe indicar esto a la capa de presentación. Aquí es donde se deben integrar los algoritmos implementados en la primera parte del laboratorio.
- **Corregir_mensaje:** si el algoritmo tiene la capacidad de corregir los errores detectados, debe corregirlos.

4. RUIDO

- **Aplicar_ruido:** el ruido no es una capa como tal, pero a fin de simular interferencias, se tratará como una capa del lado del emisor y se aplicará ruido a la trama proporcionada por la capa de enlace. La forma de determinar si cada bit sufre un flip se basará en cierta probabilidad expresada en errores por bits transmitido (por ejemplo, 1/100 es un error por cada 100 bits). Esta tasa debe ser solicitada el momento de enviar un mensaje. Recuerde que la información de redundancia (p. ej., bits de paridad) también está sujeta al ruido.

5. TRANSMISION

- **Enviar información:** envía la trama de información a través de sockets mediante el puerto elegido.
- **Recibir información:** recibe la trama de información a través de sockets mediante el puerto elegido. El receptor siempre debe estar “escuchando” en el puerto elegido a la espera de recibir data.

La aplicación del lado del emisor debe implementarse en un lenguaje de programación distinto del del receptor.

b. Algoritmos a implementar

Se deberán implementar al menos dos algoritmos (uno por integrante). De estos algoritmos, como mínimo, uno debe ser de corrección de errores y otro de detección de errores. Se deben implementar tanto el emisor como el receptor para cada algoritmo e integrarlos en la capa de enlace de la arquitectura solicitada.

Lista de algoritmos sugeridos (pueden implementar otros):

- Corrección de errores
 - Código de Hamming
 - Para cualquier $código(n,m)$ que cumpla $(m + r + 1) \leq 2^r$
 - Códigos convolucionales (algoritmo de Viterbi)
 - Para cualquier trama de longitud k . La tasa de código es $m:1$ (por cada bit de entrada, salen m bits de salida).
- Detección de errores
 - Fletcher checksum
 - Para cualquier trama de longitud k , con bloques de 8, 16 o 32 (las 3

opciones, configurable). k debe corresponder al bloque utilizado (mayor que el bloque, se agregan 0s de padding en caso el mensaje sea menor).

- CRC-32

- Para cualquier trama de longitud n , $M_n(x)$, y el polinomio estándar para CRC-32 (uno de 32 bits, investigar cual es), donde $n > 32$ (o padding si es menor a eso).

c. Pruebas

Utilizando los algoritmos implementados, realizar pruebas de envío y recepción que evidencien su funcionamiento. Para estas pruebas, cada grupo deberá elegir cómo las realizará y generar gráficas que respalden estos datos. La cantidad y el contenido de las gráficas quedan a discreción del grupo; no obstante, deben ser realizadas variando el tamaño de las cadenas enviadas, la probabilidad de error, el algoritmo utilizado y el *overhead* (cantidad de información extra que debe ser enviada como redundancia para que el algoritmo sea efectivo).

Algunas preguntas que pueden ayudar a la discusión son:

- ¿Qué algoritmo tuvo un mejor funcionamiento?
- ¿Qué algoritmo es más flexible para aceptar mayores tasas de errores?
- ¿Cuándo es mejor utilizar un algoritmo de detección errores en lugar de uno de corrección de errores?

4 Reporte

Al finalizar el laboratorio debe realizarse un reporte **grupal** donde se incluyan las siguientes secciones:

1. Nombres y carnés
2. Título de la práctica
3. Descripción de la práctica
4. Resultados
5. Discusión
6. Comentario grupal sobre el tema (opcional)
7. Conclusiones
8. Citas y Referencias

d. Rúbrica de evaluación

- Implementación de la arquitectura: 20%
- Implementación de algoritmo de detección, y comunicación con receptor: 30%
- Reporte: 50%
 - Formato: 5%
 - Pruebas: 15%
 - Discusión: 20%
 - Conclusiones: 10%

** La asistencia y participación son obligatorias; una ausencia injustificada anula la nota del laboratorio

** Se debe cuidar el formato y ortografía del reporte

e. Entregar en Canvas

- **Todo el código** involucrado y cualquier elemento para su compilación (makefiles, etc).
- **Link a su repositorio**, el cual si es privado debe otorgar permisos de acceso al profesor y auxiliares
 - El **repositorio debe tener el reporte** en formato PDF.